

S2S-05: Settings and Configuration

Import

Series: S2S | **Notebook:** 5 of 12 | **Created:** March 2026 | **Last Updated:** 03/23/2026

Overview

With the source configuration exported (S2S-02) and SSO/user access configured (S2S-03), this notebook covers importing configuration into the target tenant using Monaco or Terraform, handling entity ID remapping, and validating the import. IAM policies are deployed last — after the resources they govern exist.

Table of Contents

1. [Import Strategy](#)
 2. [Monaco Deploy Workflow](#)
 3. [Terraform Apply Workflow](#)
 4. [Entity ID Remapping](#)
 5. [Import Validation](#)
-

Prerequisites

Requirement	Details
Exported Configuration	From S2S-02 (Monaco export or Terraform HCL)
Target Tenant	Provisioned with SSO configured (S2S-03)
OAuth Client	Created in target tenant with required scopes
Terraform	Only if managing IAM policies (the one Terraform-exclusive resource)

1. Import Strategy

Deployment Order

Configuration must be deployed in dependency order — resources that other resources reference must exist first. IAM policies are deployed **last** because they restrict access to resources that must already exist.

Monaco note: `monaco deploy` resolves dependencies automatically within a single run. The order below is for reference and for teams using Terraform's ordered apply.

Data Foundation — Deploy First

Order	Category	Dependencies	Monaco Type
1	Grail buckets	None — data routing needs these first	bucket
2	Enrichment rules (K8s labels → Grail tags)	None — tags feed everything downstream	settings
3	OpenPipeline processing rules	Buckets must exist as routing targets	openpipeline
4	Segments	None	segment

Gen2 (Classic) Configuration

Order	Category	Dependencies	Monaco Type
5	Management zones	None	api / settings
6	Auto-tags	None	api / settings
7	Request attributes	None	api / settings
8	Calculated service metrics	None	api
9	Custom services and detection rules	None	api / settings
10	Application detection rules	None	api / settings
11	Conditional naming rules	None	api
12	Anomaly detection settings	None	settings
13	Credential vault entries	None	api

Monitoring & Alerting

Order	Category	Dependencies	Monaco Type
14	Alerting profiles	None	api / settings
15	Notification rules + integrations	Alerting profiles	api / settings
16	Maintenance windows	Management zones (for scope)	api / settings
17	SLO definitions	Tags, management zones	slo-v2
18	Synthetic monitors	Notification rules, credential vault	api

User-Facing Configuration

Order	Category	Dependencies	Monaco Type
19	Classic dashboards	Management zones, auto-tags	api
20	Gen3 dashboards and notebooks	SLOs, tags, segments	document
21	Workflows and automations	Notification rules, SLOs	automation

Governance — Deploy Last

Order	Category	Dependencies	Tool
22	IAM groups	All resources above exist	Terraform only
23	IAM policies	All resources above exist	Terraform only
24	IAM policy bindings	Groups + policies	Terraform only

Why IAM last? IAM policies use `WHERE` clauses that reference schemas, buckets, and security contexts (e.g., `WHERE storage:bucket.name = "app_logs"`). If those resources don't exist yet, you can't validate that the policies are correct. Deploy the resources first, then lock them down with IAM.

2. Monaco Deploy Workflow

Monaco handles the entire import (except IAM) in a single command:

```
# Update manifest to point at target tenant
# manifest.yaml → change environment URL and token to target values

# Validate before deploying
monaco validate manifest.yaml

# Dry run – preview what will change
monaco deploy manifest.yaml --environment target --dry-run

# Deploy everything (Monaco resolves dependency order automatically)
monaco deploy manifest.yaml --environment target
```

Monaco deploys all 7 non-IAM types in one pass: `settings`, `document`, `automation`, `bucket`, `segment`, `slo-v2`, `openpipeline`, and classic `api`.

Then Add IAM with Terraform (if needed)

```
# Only needed for IAM policies – everything else is handled by Monaco above
cd terraform/iam/

terraform init
terraform plan    # Review the IAM policies before applying
terraform apply   # Deploy groups, policies, and bindings
```

3. Terraform Apply Workflow

If using Terraform for the entire migration instead of Monaco:

```
# 1. Validate configuration
terraform validate

# 2. Preview changes (critical – review before applying!)
terraform plan -out=migration.tfplan

# 3. Apply resources (Terraform resolves dependencies via resource references)
terraform apply migration.tfplan

# 4. Apply IAM last (separate state or targeted apply)
terraform apply -target=dynatrace_iam_group.all
terraform apply -target=dynatrace_iam_policy.all
terraform apply -target=dynatrace_iam_policy_bindings_v2.all

# 5. Verify state
terraform state list | wc -l
```

Handling Naming Collisions (Consolidation)

When merging multiple tenants, prefix resources to avoid collisions:

```
variable "source_prefix" {
  description = "Source tenant identifier for naming"
  type        = string
  default     = "us-east" # or "emea", "apac", etc.
}

resource "dynatrace_alerting" "critical" {
  name = "${var.source_prefix} - Critical Alerts"
  # ... rest of configuration
}
```

4. Entity ID Remapping

Entity IDs (e.g., `HOST-1A2B3C4D`, `SERVICE-5E6F7A8B`) are unique per tenant. Any configuration that references entity IDs must be updated.

Where Entity IDs Appear

Configuration	Example Reference	How to Fix
Dashboard tile filters	<code>entityId("HOST-1A2B3C4D")</code>	Replace with tag-based filter or look up new ID
SLO metric expressions	<code>type(SERVICE),entityId(SERVICE-5E6F7A8B)</code>	Replace with <code>type(SERVICE),tag("app:checkout")</code>
Notification rule filters	<code>entityId</code> in tag filter	Replace with tag-based filter
Management zone rules	Entity-specific rules	Replace with tag or property rules

Best Practice: Replace Entity IDs with Tags

```
# BEFORE (entity ID – breaks on migration)
filter = "type(SERVICE),entityId(SERVICE-5E6F7A8B)"

# AFTER (tag-based – survives migration)
filter = "type(SERVICE),tag(app:checkout),tag(env:production)"
```

Recommendation: Before migration, audit all dashboards and SLOs for hardcoded entity IDs. Convert to tag-based filters. This makes the configuration portable across tenants.

Entity ID Lookup in Target

After agents report to the target tenant, look up new entity IDs:

```
// Find a service by name in the target tenant
fetch dt.entity.service
| filter contains(entity.name, "checkout")
| fields entity.name, id
| limit 10
```

5. Import Validation

Post-Import Checklist

Category	Validation	DQL/Command
Settings count	Source count matches target count	Settings API: compare <code>totalCount</code> per schema
Dashboards	All dashboards imported	Document API: compare counts
SLOs	All SLOs created and evaluating	SLO API: check status
Notification rules	Rules created and active	Settings API: check <code>builtin:alerting.profile</code>
Enrichment rules	Tags populating on new data	DQL: check <code>dt.security_context</code> on logs
Workflows	Workflows created and enabled	Automation API: check status

Next Steps

Continue to **S2S-06: Cloud Integration Migration** to migrate cloud provider connections.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.